

Atividade Integradora: Dicionário

Disciplina: Redes de Computadores

Alunos: Henrique Leão Paim e Rhuan Victor Sousa Azevedo

Tema Selecionado: Consulta de Dicionário

1. Documentação do Protocolo de Aplicação

A comunicação foi projetada para ser simples, textual e síncrona.

Nome do protocolo	DICTP/1.0 (Dictionary Protocol)
Transporte utilizado	TCP. A confiabilidade do TCP é necessária para garantir que textos longos (definições) cheguem ínteros e na ordem correta, sem perda de pacotes.
Porta padrão	5555
Formato das mensagens	Texto plano codificado em UTF-8. Toda mensagem, enviada ou recebida, deve terminar obrigatoriamente com uma quebra de linha (\n).
Comandos (Cliente)	SEARCH <termo>: Busca a definição de uma palavra. LIST: Solicita a lista de palavras disponíveis no dicionário. QUIT: Solicita o encerramento da conexão.
Respostas (Servidor)	FOUND <definição>: Retornado quando a busca tem sucesso. NOTFOUND: Retornado quando o termo não existe. LISTING <palavra1, palavra2, ...>: Retorna os termos cadastrados. BYE: Confirmação de encerramento.
Erros	ERROR Invalid Command: Enviado quando o servidor não reconhece a sintaxe do comando recebido.
Encerramento	O cliente envia QUIT. O servidor responde BYE e, em seguida, encerra ativamente o socket de comunicação. O cliente, ao receber BYE, também fecha seu socket.

2. Código do Servidor (server.py)

O servidor utiliza a biblioteca padrão threading para permitir que múltiplos clientes se conectem e consultem o dicionário simultaneamente. Ele escuta em 0.0.0.0 para aceitar conexões da rede local.

```
import socket
import threading

HOST = '0.0.0.0'
PORT = 5555

# Banco de dados em memória
DICCIONARIO = {
    "socket": "Ponto final de uma conexao de rede bidirecional.",
    "protocolo": "Conjunto de regras que governa a comunicacao entre computadores.",
    "tcp": "Protocolo de Controle de Transmissao, orientado a conexao e confiavel.",
    "hardware": "Parte fisica do computador.",
    "foco": "Capacidade de manter a atencao concentrada em um objetivo."
}

def handle_client(conn, addr):
    print(f"[+] Nova conexão de {addr}")
    with conn:
        while True:
            try:
                data = conn.recv(1024)
                if not data:
                    break

                mensagem = data.decode('utf-8').strip()
                partes = mensagem.split(' ', 1)
                comando = partes[0].upper()
```

```

        if comando == 'SEARCH' and len(partes) > 1:
            termo = partes[1].lower()
            definicao = DICIONARIO.get(termo)
            if definicao:
                resposta = f"FOUND {definicao}\n"
            else:
                resposta = "NOTFOUND\n"

        elif comando == 'LIST':
            termos = ", ".join(DICIONARIO.keys())
            resposta = f"LISTING {termos}\n"

        elif comando == 'QUIT':
            conn.sendall("BYE\n".encode('utf-8'))
            print(f"[-] Conexão encerrada ativamente por
{addr}")

            break

        else:
            resposta = "ERROR Invalid Command\n"

            conn.sendall(resposta.encode('utf-8'))

    except ConnectionResetError:
        break
    except Exception as e:
        print(f"[!] Erro com {addr}: {e}")
        break

print(f"[-] Cliente {addr} desconectado.")

def start_server():
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as
server_socket:
        server_socket.bind((HOST, PORT))
        server_socket.listen(5)
        print(f"[*] Servidor DICTP/1.0 escutando em {HOST}:{PORT}")

```

```

        try:
            while True:
                conn, addr = server_socket.accept()
                thread = threading.Thread(target=handle_client,
args=(conn, addr))
                thread.start()
                print(f"[*] Conexões ativas: {threading.active_count()
- 1}")
            except KeyboardInterrupt:
                print("\n[*] Servidor encerrado.")

if __name__ == '__main__':
    start_server()

```

3. Código do Cliente (client.py)

O cliente é projetado para operar via terminal, conectando-se ao IP fornecido pelo usuário.

```

import socket
import sys

PORT = 5555

def start_client(server_ip):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as
client_socket:
        try:
            client_socket.settimeout(10.0) # Timeout para evitar
travamento
            client_socket.connect((server_ip, PORT))
            print(f"[*] Conectado ao servidor {server_ip}:{PORT}")
            print("Comandos disponíveis: SEARCH <termo>, LIST, QUIT")

            while True:
                comando = input("DICTP> ").strip()

```

```
        if not comando:
            continue

        # Adiciona o \n exigido pelo protocolo
        mensagem = comando + "\n"
        client_socket.sendall(mensagem.encode('utf-8'))

        resposta =
client_socket.recv(1024).decode('utf-8').strip()
        print(f"Servidor: {resposta}")

        if resposta == "BYE":
            break

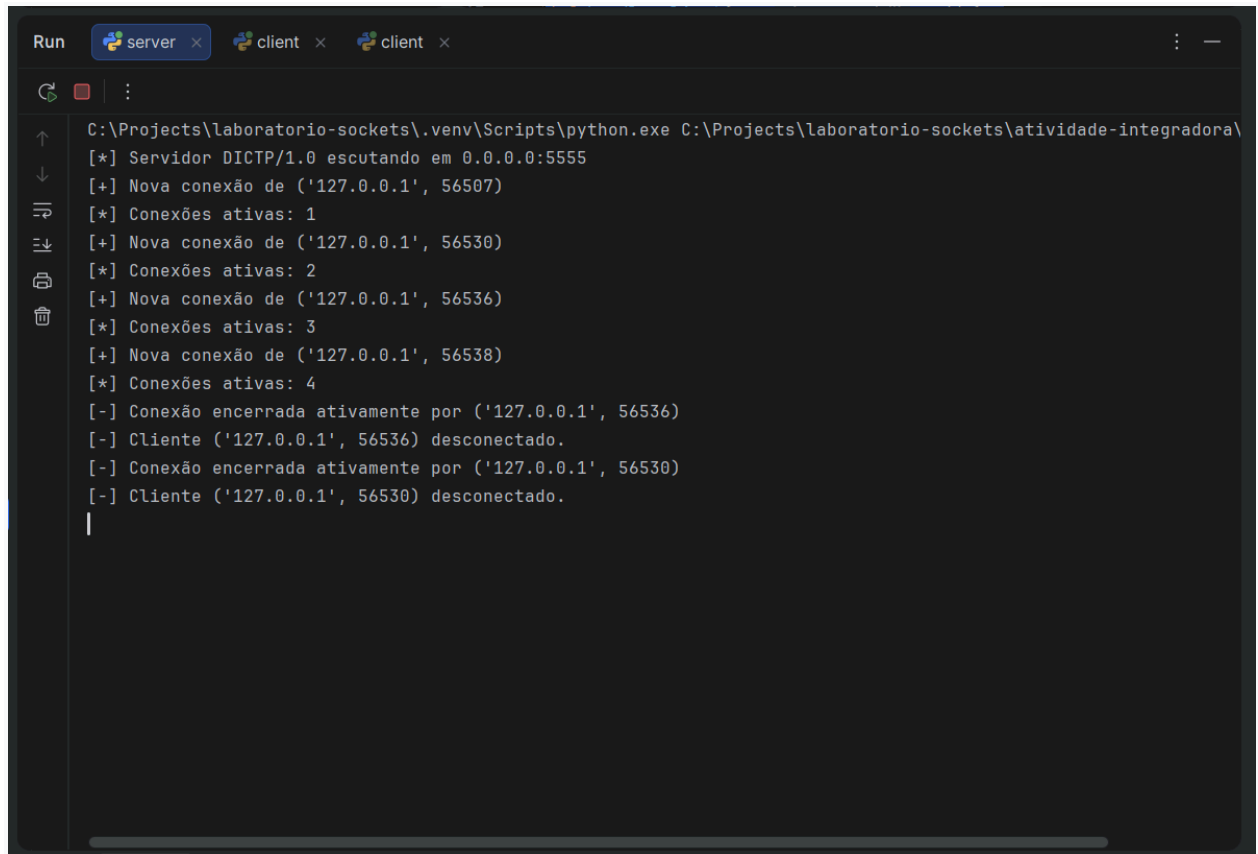
    except ConnectionRefusedError:
        print("[!] Erro: Conexão recusada. O servidor está
rodando?")
    except socket.timeout:
        print("[!] Erro: Tempo de conexão esgotado.")
    except Exception as e:
        print(f"[!] Erro inesperado: {e}")

if __name__ == '__main__':
    ip = input("Digite o IP do servidor (ex: 192.168.1.10 ou
127.0.0.1): ").strip()
    start_client(ip)
```

4. Evidências de Teste (Cenário de Execução)

O sistema foi estruturado para ser executado em rede local (LAN). O servidor roda na máquina 192.168.0.15 e o cliente conecta a partir de outra máquina na mesma rede.

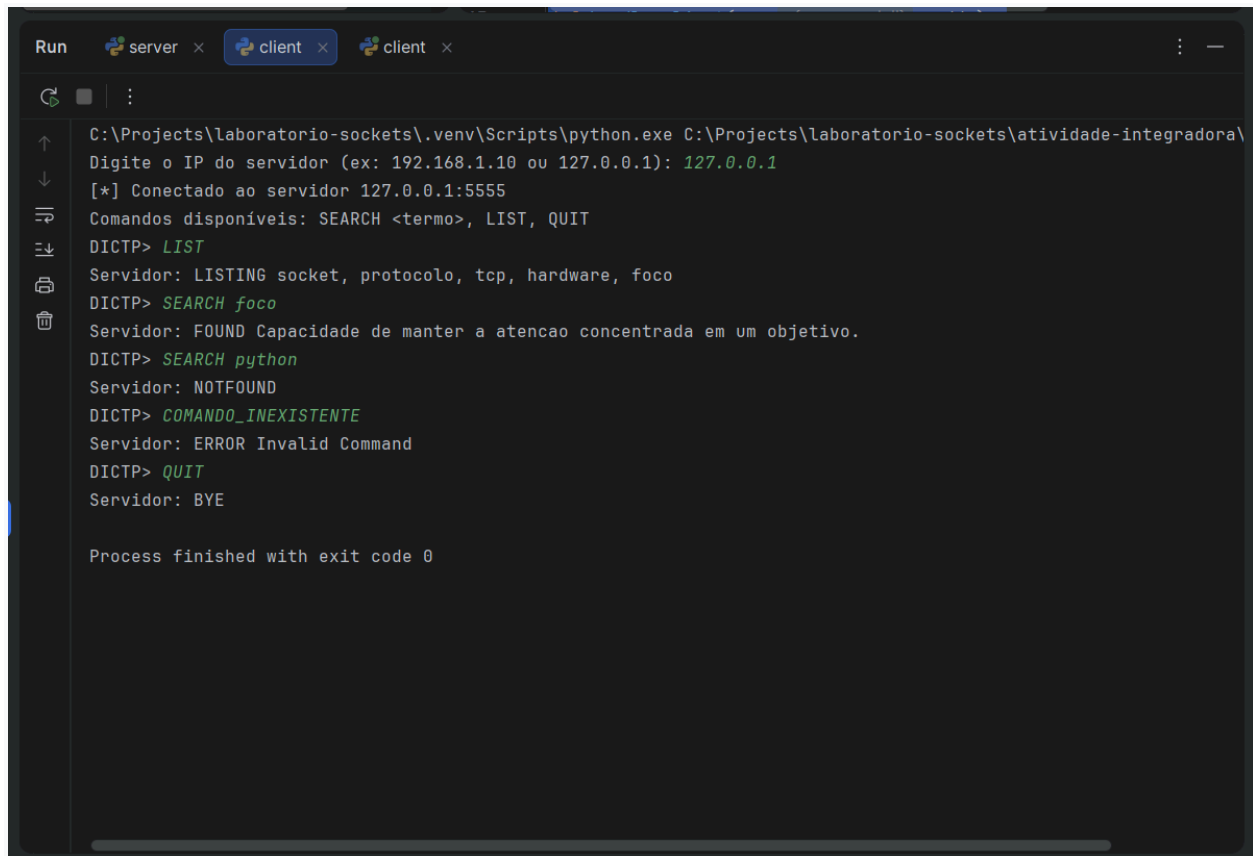
Logs do Servidor:



The screenshot shows a terminal window titled "Run" with three tabs: "server", "client", and "client". The terminal output displays the following logs:

```
C:\Projects\laboratorio-sockets\.venv\Scripts\python.exe C:\Projects\laboratorio-sockets\atividade-integradora\
[*] Servidor DICTP/1.0 escutando em 0.0.0.0:5555
[+] Nova conexão de ('127.0.0.1', 56507)
[*] Conexões ativas: 1
[+] Nova conexão de ('127.0.0.1', 56530)
[*] Conexões ativas: 2
[+] Nova conexão de ('127.0.0.1', 56536)
[*] Conexões ativas: 3
[+] Nova conexão de ('127.0.0.1', 56538)
[*] Conexões ativas: 4
[-] Conexão encerrada ativamente por ('127.0.0.1', 56536)
[-] Cliente ('127.0.0.1', 56536) desconectado.
[-] Conexão encerrada ativamente por ('127.0.0.1', 56530)
[-] Cliente ('127.0.0.1', 56530) desconectado.
|
```

Terminal do Cliente (192.168.0.20):



```
Run server x client x client x
C:\Projects\laboratorio-sockets\.venv\Scripts\python.exe C:\Projects\laboratorio-sockets\atividade-integradora\
Digite o IP do servidor (ex: 192.168.1.10 ou 127.0.0.1): 127.0.0.1
[*] Conectado ao servidor 127.0.0.1:5555
Comandos disponiveis: SEARCH <termo>, LIST, QUIT
DICTP> LIST
Servidor: LISTING socket, protocolo, tcp, hardware, foco
DICTP> SEARCH foco
Servidor: FOUND Capacidade de manter a atencao concentrada em um objetivo.
DICTP> SEARCH python
Servidor: NOTFOUND
DICTP> COMANDO_INEXISTENTE
Servidor: ERROR Invalid Command
DICTP> QUIT
Servidor: BYE

Process finished with exit code 0
```

5. Limitações e Possíveis Melhorias

- **Persistência de Dados:** O dicionário está alocado estaticamente na memória RAM (em um objeto dict do Python). Se o servidor for reiniciado, alterações seriam perdidas. Uma melhoria clara seria conectar o servidor a um banco de dados relacional.
- **Segurança e Criptografia:** Os dados trafegam em texto plano via rede. Não há uso de TLS (Transport Layer Security). Uma pessoa inspecionando o tráfego com um sniffer poderia ler todo o conteúdo das buscas.
- **Problema do Fragmento TCP (Stream):** O método `recv(1024)` do servidor presume que receberá a mensagem inteira em um único bloco. Em redes com alta latência, a mensagem pode ser fatiada, exigindo a implementação de um buffer no servidor que acumule bytes até encontrar o caractere terminador `\n` antes de processar o comando.